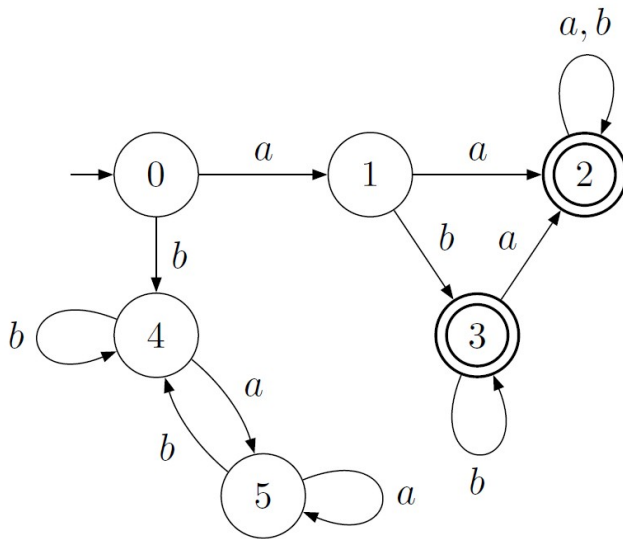




G52LAC (COMP2040)

Languages and Computation

Lecture 9: PDAs & CFGs

Tomas Maul
School of Computer Science
University of Nottingham Malaysia Campus

Outline

- Quick recap. Any questions?
- Equivalence of PDAs and CFGs.
- Derivation Trees and Ambiguity.

Outline

- Quick recap. Any questions?
- Equivalence of PDAs and CFGs.
- Derivation Trees and Ambiguity.

Empty stack & final state

- Acceptance by “empty stack” and “final state” are equivalent in the sense that:
 - (from Hopcroft et al) “A language L has a PDA that accepts it by final state if and only if L has a PDA that accepts it by empty stack”.
- However, for a given PDA P , the languages that P accepts by “final state” are generally different from those that P accepts by “empty stack”.
- In the next few slides we will show you how to convert a PDA P that accepts a language L via “empty stack” into a PDA P' that accepts the same language L via “final state”, and vice versa.

Empty Stack to Final State: Informally

- Basic idea: when the stack is empty, go to an accepting state.
- Summary:
 - Set the initial state (q_0) to not be initial.
 - Add a new initial state (p_0).
 - Add a final state (p_f).
 - Add a transition $(\varepsilon, X_0 / Z_0 X_0)$ from p_0 to q_0 .
 - Add transitions $(\varepsilon, X_0 / \varepsilon)$ from all states (except p_0 and p_f) to p_f .
 - X_0 is the new initial stack symbol.
- If p_0 , p_f or X_0 are already in use, then use different symbols!

Empty Stack to Final State: Formally

Given an *Empty-Stack* PDA

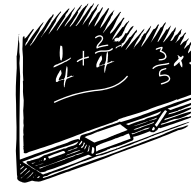
$$P_N = (Q_N, \Sigma, \Gamma_N, \delta_N, q_0, Z_0)$$

an equivalent *Final-State* PDA is defined:

$$P_F = (Q_N \cup \{p_0, p_f\}, \Sigma, \Gamma_N \cup \{X_0\}, \delta_F, p_0, X_0, \{p_f\})$$

where δ_F is all the transitions in δ_N plus:

$$\begin{aligned} \delta_F(p_0, \varepsilon, X_0) &= \{(q_0, Z_0 X_0)\} \\ \delta_F(q, \varepsilon, X_0) \mid q \in Q_N &= \{(p_f, \varepsilon)\} \end{aligned}$$

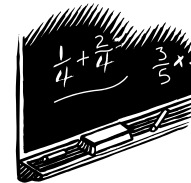


Page 237, Fig. 6.4

Final State to Empty Stack: Informally

- Basic idea: when in a final state, empty the stack.
- Summary:
 - Set the initial state (q_0) to not be initial.
 - Add a new initial state (p_0).
 - Add a new state (p).
 - Add a transition ($\varepsilon, X_0 / Z_0 X_0$) from p_0 to q_0 .
 - Add transitions ($\varepsilon, y / \varepsilon$) from p and all final states to p ($\forall y \in \Gamma_F \cup \{X_0\}$).
 - Set all states to be not accepting.
 - X_0 is the new initial stack symbol.
- If p_0 , p or X_0 are already in use, then use different symbols!

Final State to Empty Stack: Formally



Page 240, Fig. 6.7

Given an *Final-State* PDA

$$P_F = (Q_F, \Sigma, \Gamma_F, \delta_F, q_0, Z_0, F)$$

an equivalent *Empty-Stack* PDA is defined:

$$P_N = (Q_F \cup \{p_0, p\}, \Sigma, \Gamma_F \cup \{X_0\}, \delta_N, p_0, X_0)$$

where δ_N is all the transitions in δ_F plus:

$$\begin{aligned} \delta_N(p_0, \varepsilon, X_0) &= \{(q_0, Z_0 X_0)\} \\ \delta_N(q, \varepsilon, -) \mid q \in (F \cup \{p\}) &= \{(p, \varepsilon)\} \end{aligned}$$

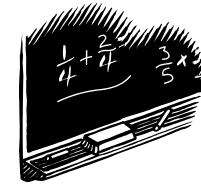
CFG to Empty-Stack PDA: Informally

Basic idea:

- Push all words (non-deterministically) in the language onto the stack using the CFG productions.
- Then match the input word against the stack contents, popping the stack if they match.
- If both the stack and word are empty, then the word must be in the language.

CFG to Empty-Stack PDA: Formally

Given a Context-Free Grammar



Notes ...

$$G = (N, \Sigma, P, S)$$

an equivalent *Empty-Stack* PDA is defined:

$$P_N = (\{q_0\}, \Sigma, N \cup \Sigma, \delta, q_0, S)$$

where:

$$\begin{aligned}\delta(q_0, \varepsilon, A) &= \{(q_0, \beta) \mid (A \rightarrow \beta) \in P\} && \text{for all } A \in N \\ \delta(q_0, a, a) &= \{(q_0, \varepsilon)\} && \text{for all } a \in \Sigma \\ \delta(q_0, -, -) &= \emptyset\end{aligned}$$

Deterministic Pushdown Automata (DPDAs)

- Informally: A deterministic PDA is one that has **at most** one possible transition at any point.
- Formally: A PDA $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ or $(Q, \Sigma, \Gamma, \delta, q_0, Z_0)$ is deterministic iff:

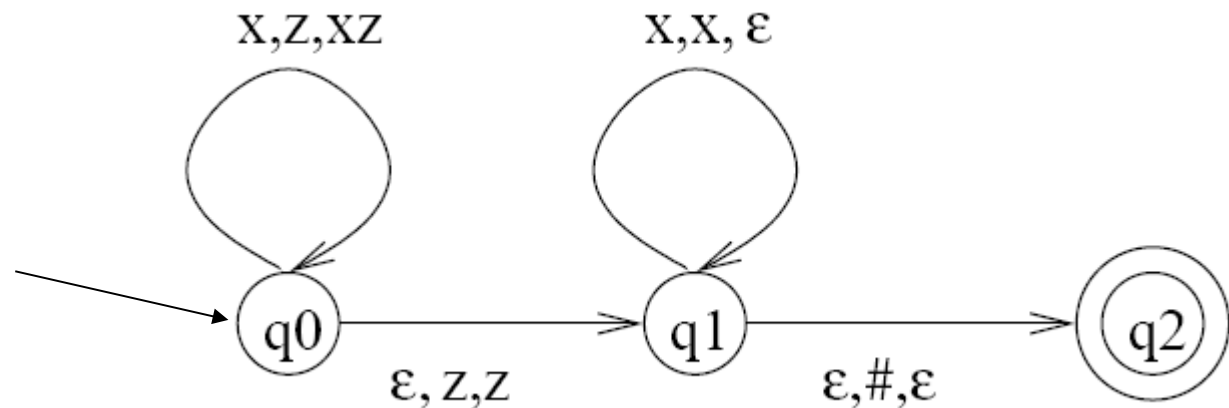
$$\forall q \in Q, x \in \Sigma, z \in \Gamma. |\delta(q, x, z)| + |\delta(q, \varepsilon, z)| \leq 1$$

Deterministic Pushdown Automata (DPDAs)

Recall the language:

$$L = \{ww^R \mid w \in \{0, 1\}^*\}$$

... and the PDA that can generate it:



Problem:

Unlike the situation for finite automata, there is in general no way to translate a nondeterministic PDA into a deterministic one. Indeed, there is no DPDA that recognizes the language L ! **Nondeterministic PDAs are more powerful than deterministic PDAs.**

Deterministic Pushdown Automata (DPDAs)

Challenge:

Q1: Why can't a DPDA generate this language?

$$L = \{ww^R \mid w \in \{0, 1\}^*\}$$

Q2: Based on your answer to the above question Q1, what small modification to L can you make in order to allow a DPDA to generate it?

Deterministic Pushdown Automata (DPDAs)

Challenge:

Q1: Why can't a DPDA generate this language?

$$L = \{ww^R \mid w \in \{0, 1\}^*\}$$

A1: How does it know when it has reached the middle of the word?
The non-deterministic PDA can constantly guess that it has reached the middle.

Q2: Based on your answer to the above question Q1, what small modification to L can you make in order to allow a DPDA to generate it?

A2: $L' = \{w\$w^R \mid w \in \{0, 1\}^*\}$

Deterministic Pushdown Automata (DPDAs)

Challenge:

Q1: Why can't a DPDA generate this language?

$$L = \{ww^R \mid w \in \{0, 1\}^*\}$$

A1: How does it know when it has reached the middle of the word? The non-deterministic PDA can constantly guess that it has reached the middle.

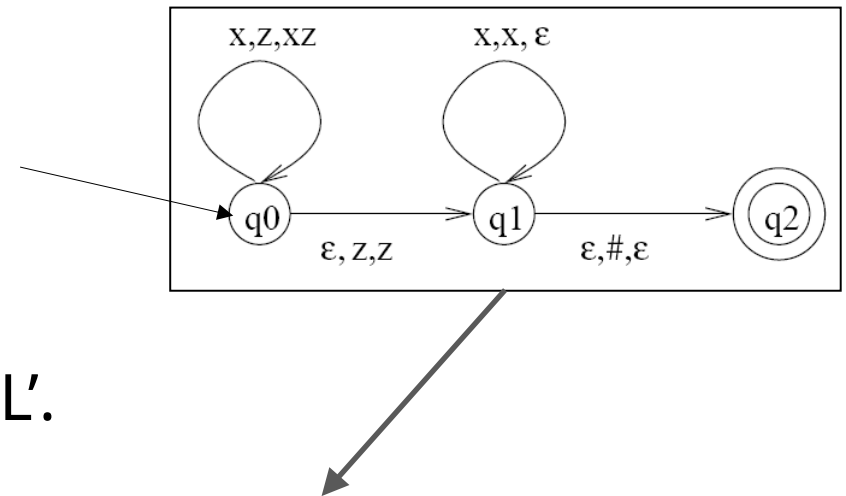
Q2: Based on your answer to the above question Q1, what small modification to L can you make in order to allow a DPDA to generate it?

A2: $L' = \{w\$w^R \mid w \in \{0, 1\}^*\}$

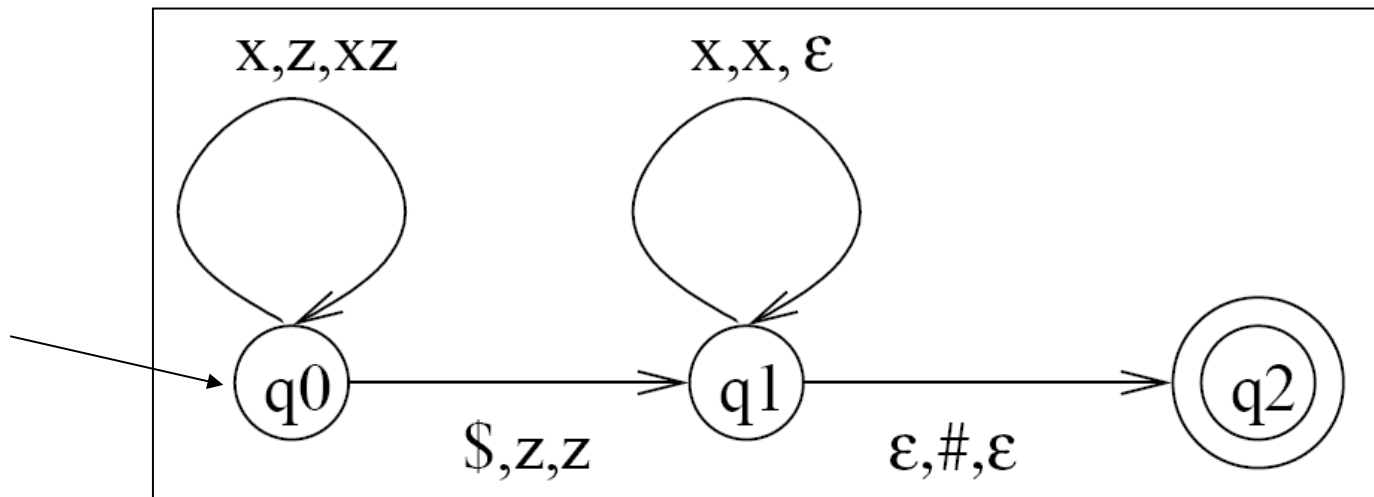
Give a DPDA that can generate L' .

Deterministic Pushdown Automata (DPDAs)

$$L' = \{w\$w^R \mid w \in \{0, 1\}^*\}$$



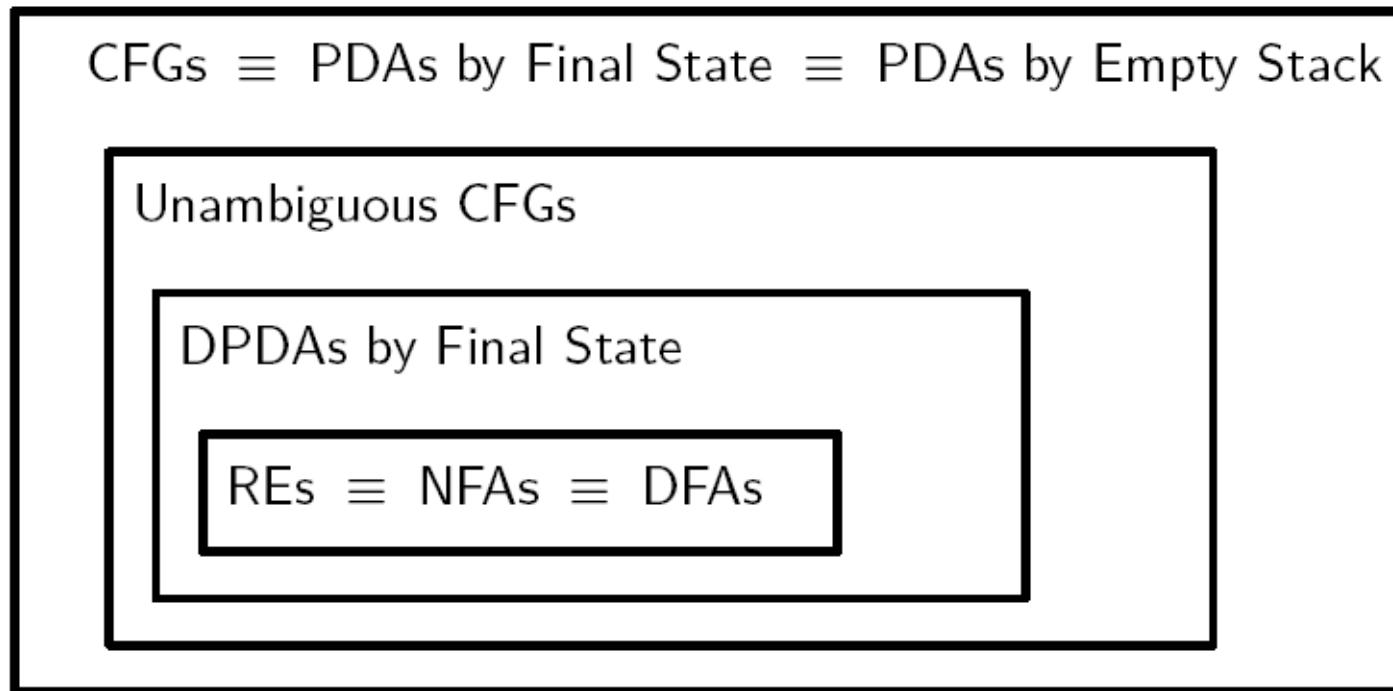
Give a DPDA that can generate L' .



Efficient, but less Expressive

- DPDAs can be implemented more efficiently than non-deterministic PDAs.
- Unfortunately, the set of languages described by DPDAs is a **strict subset** of the set of languages described by PDAs.
- Example: A language of palindromes
 - words that are the same when reversed
 - no way to know when the middle of the word is reached
 - thus doesn't know whether to push or pop
- However, most Context-Free Languages of practical importance can be described by DPDAs.

Context-Free Language Hierarchy



- Empty-Stack DPDAs are less expressive than Final-State DPDAs; they cannot even recognise all Regular Languages.
- All DPDA languages have unambiguous CFGs, but not all unambiguous grammars generate DPDA languages.

Recommended Reading

- Introduction to Automata Theory, Languages, and Computation (3rd edition), pages 237–246 & 252–258
- G52MAL Lecture Notes, pages 41–43

Outline

- Quick recap. Any questions?
- Equivalence of PDAs and CFGs.
- Derivation Trees and Ambiguity.

Derivation Trees

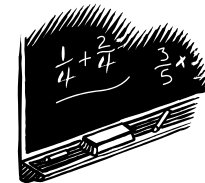
- The structure of a derivation can be drawn as a tree.
- This is called a **derivation tree** or **parse tree**.
- The string of **leaf** labels read from left to right is called the **yield** of the tree.
- For a CFG G , a string α is the yield of a derivation tree iff α is a sentential form of G .

Formal Definition: Derivation Tree

A tree is a **derivation tree** for a CFG $G = (N, \Sigma, P, S)$ iff:

- 1 The label of the root node is S .
- 2 The label of every node is in $(N \cup \Sigma \cup \{\varepsilon\})$.
- 3 The label of every interior node is in N .
- 4 Only leaf nodes that are the only child of their parent can have label ε .
- 5 If a node has label A , and its child nodes have labels $x_1, x_2, \dots, x_n$ (from left to right), then $A \rightarrow x_1 x_2 \dots x_n$ is a production in P .

Ambiguity



Notes ...

- A CFG G is **ambiguous** if there exists **at least one** word $w \in L(G)$ for which there is **more than one** derivation tree.
- Equivalently, G is ambiguous if there are at least two different:
 - **leftmost** derivations for w
or
 - **rightmost** derivations for w
- Because each derivation tree corresponds to **exactly one** leftmost derivation and **exactly one** rightmost derivation.

Ambiguity: Common Mistakes

- The two trees (or derivations) must be for the **same word**.
- Just having two different derivations for a word **does not** demonstrate ambiguity. They must be **both leftmost** or **both rightmost**.

Example of Ambiguity

Deriving: $\text{id} + \text{id} * \text{id}$

PRODUCTIONS

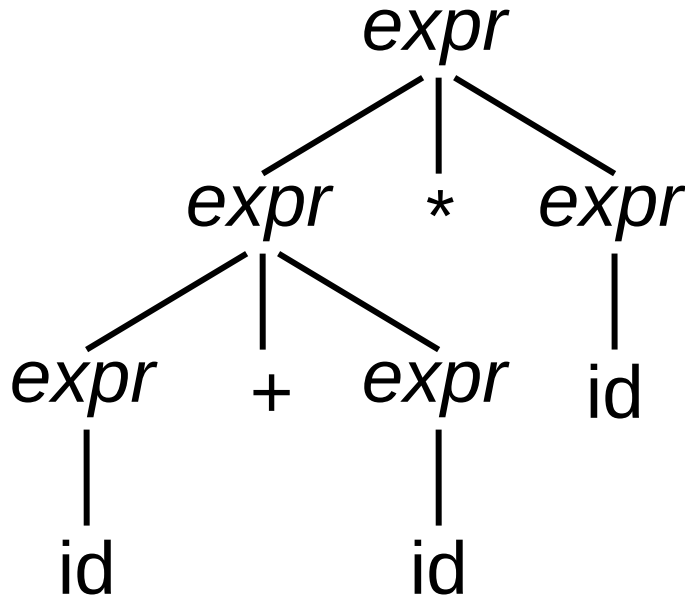
$\text{expr} \rightarrow \text{expr} + \text{expr}$

$\text{expr} \rightarrow \text{expr} * \text{expr}$

$\text{expr} \rightarrow '(' \text{expr} ')'$

$\text{expr} \rightarrow '-' \text{expr}$

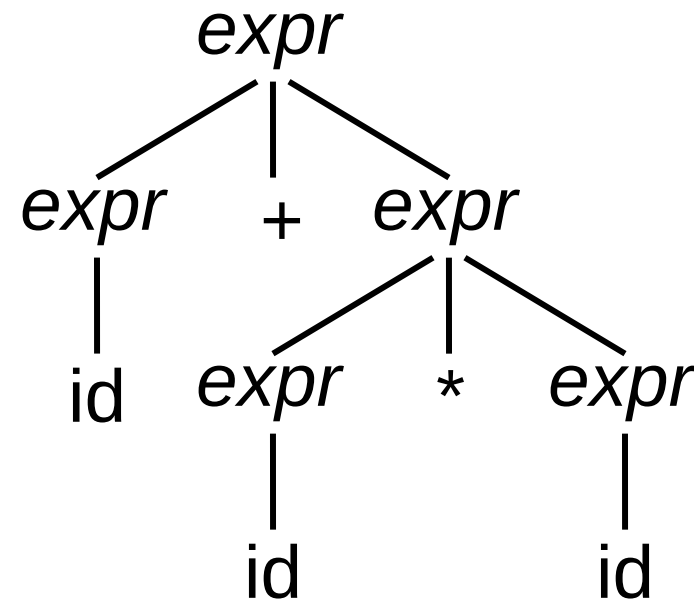
$\text{expr} \rightarrow \text{id}$



Interpreted as $(\text{id} + \text{id}) * \text{id}$

If $\text{id} = 3$ then we have

$$(3+3)*3 = 18$$



Interpreted as $\text{id} + (\text{id} * \text{id})$

If $\text{id} = 3$ then we have

$$3+(3*3) = 12$$

Recommended Reading

- Introduction to Automata Theory, Languages, and Computation (3rd edition), pages 183–186 & 207–208
- G52MAL Lecture Notes, pages 35–37

Acknowledgements

- Thorsten Altenkirch.
- Neil Sculthorpe.
- Henrik Nilsson.
- V.P. Kallimani.